

Applied Machine Learning — Week 3

Bias–Variance · Decision Trees · Bagging & Random Forests · Boosting
Comprehensive Revision Notes

Contents

I	Bias–Variance Decomposition	2
1	Setup & Notation	2
2	The Decomposition	2
2.1	Full Derivation	2
2.2	Interpretation	3
2.3	Diagnosing the Regime	3
II	Decision Trees (CART)	4
3	Concept	4
4	The CART Algorithm	4
4.1	Stopping Criteria	4
4.2	Properties of Decision Trees	4
III	Bagging & Random Forests	6
5	Motivation: Reducing Variance	6
6	The Bootstrap	6
7	Bagging Algorithm	6
7.1	Out-of-Bag (OOB) Error	6
8	Random Forests	7
IV	Boosting	8
9	Motivation: Reducing Bias	8
10	Gradient Descent in Function Space	8
11	Gradient Boosted Regression Trees (GBRT)	8

12 AdaBoost	9
12.1 Setting	9
12.2 Finding the Best Weak Learner	9
12.3 Closed-Form Step Size	9
12.4 Weight Update	10
12.5 Training Loss Bound	10
12.6 Full AdaBoost Algorithm	10
 V Exam-Style Questions & Lecture Exercises	 12
13 Exam-Style Questions	12
14 Lecture Exercises & Worked Solutions	13
14.1 Exercise 1: Second Cross-Term Vanishes	13
14.2 Exercise 2: Can Every Dataset Have a Pure Tree?	13
14.3 Exercise 3: Why Not Stop When No Split Improves Impurity?	13

Fei Gao

Part I

Bias–Variance Decomposition

Setup & Notation

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \sim p(X, Y)$ be a training set. Let $\mathcal{A}$ be a learning algorithm that produces predictor $h_{\mathcal{D}} = \mathcal{A}(\mathcal{D})$.

Key Quantities

- **Expected label:** $\bar{y}(x) = \mathbb{E}_{y|x}[Y] = \int y p(y|x) dy$. The best possible prediction at x (Bayes-optimal for squared error).
- **Expected classifier:** $\bar{h}(x) = \mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(x)] = \int h_{\mathcal{D}}(x) p(\mathcal{D}) d\mathcal{D}$. The average prediction across all possible training sets.
- **Expected test error:** $\mathbb{E}_{x,y,\mathcal{D}}[(h_{\mathcal{D}}(x) - y)^2]$. Integrates over test point (x, y) AND the random training set $\mathcal{D}$.

The Decomposition

Bias–Variance–Noise Decomposition

$$\underbrace{\mathbb{E}_{x,y,\mathcal{D}}[(h_{\mathcal{D}}(x) - y)^2]}_{\text{expected test error}} = \underbrace{\mathbb{E}_{x,\mathcal{D}}[(h_{\mathcal{D}}(x) - \bar{h}(x))^2]}_{\text{Variance}} + \underbrace{\mathbb{E}_x[(\bar{h}(x) - \bar{y}(x))^2]}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{y|x}[(\bar{y}(x) - y)^2]}_{\text{Noise}}$$

Full Derivation

Step 1: Add and subtract $\bar{h}(x)$:

$$\begin{aligned} (h_{\mathcal{D}}(x) - y)^2 &= [(h_{\mathcal{D}}(x) - \bar{h}(x)) + (\bar{h}(x) - y)]^2 \\ &= (h_{\mathcal{D}} - \bar{h})^2 + 2(h_{\mathcal{D}} - \bar{h})(\bar{h} - y) + (\bar{h} - y)^2. \end{aligned}$$

Take $\mathbb{E}_{x,y,\mathcal{D}}$. The cross-term vanishes (see proof below), leaving:

$$\mathbb{E}[(h_{\mathcal{D}} - y)^2] = \underbrace{\mathbb{E}[(h_{\mathcal{D}} - \bar{h})^2]}_{\text{Variance}} + \mathbb{E}[(\bar{h} - y)^2].$$

Step 2: Decompose the second term by adding and subtracting $\bar{y}(x)$:

$$\begin{aligned} (\bar{h} - y)^2 &= [(\bar{h} - \bar{y}) + (\bar{y} - y)]^2 \\ &= (\bar{h} - \bar{y})^2 + 2(\bar{h} - \bar{y})(\bar{y} - y) + (\bar{y} - y)^2. \end{aligned}$$

The cross-term again vanishes (Exercise 1), leaving:

$$\mathbb{E}[(\bar{h} - y)^2] = \underbrace{\mathbb{E}[(\bar{h} - \bar{y})^2]}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\bar{y} - y)^2]}_{\text{Noise}}.$$

Proof: First Cross-Term = 0

$$\begin{aligned}\mathbb{E}_{x,y,\mathcal{D}}[(h_{\mathcal{D}} - \bar{h})(\bar{h} - y)] &= \mathbb{E}_{x,y}[\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}} - \bar{h}] \cdot (\bar{h} - y)] \\ &= \mathbb{E}_{x,y}[(\bar{h} - \bar{h}) \cdot (\bar{h} - y)] = 0.\end{aligned}$$

Key step: $(\bar{h} - y)$ does not depend on $\mathcal{D}$, so it factors out. Then $\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}] = \bar{h}$ by definition.

Interpretation

What Each Term Means

Variance: how much does $h_{\mathcal{D}}$ change if we train on a different dataset? High variance = the model is very sensitive to the particular training set (overfitting).

Bias²: the error of the *average* predictor $\bar{h}$, even with infinite data. Measures the inherent limitation of the model class / algorithm. High bias = underfitting.

Noise: irreducible error due to the stochasticity of y given x . Cannot be reduced by any algorithm. Depends on $p(y|x)$ and the feature representation.

Diagnosing the Regime

Detecting High Variance vs. High Bias

High variance (overfitting):

- Training error $\ll$ test error (large gap).
- Fixes: collect more data, reduce model complexity, **bagging** (this lecture!), regularisation.

High bias (underfitting):

- Training error $\approx$ test error, *both* high.
- Fixes: increase model complexity, add more features, kernels, **boosting** (this lecture!).

Connections:

Bias-variance drives the rest of Week 3 Full-depth decision trees: **low bias, high variance** $\Rightarrow$ use **bagging** (Random Forests).

Shallow (limited-depth) trees: **high bias, low variance** $\Rightarrow$ use **boosting** (AdaBoost, GBRT).

Part II

Decision Trees (CART)

Concept

Decision trees recursively partition the feature space using axis-aligned splits. Each internal node tests one feature against a threshold; each leaf provides a prediction.

Goals: (1) maximally compact trees, (2) pure leaves (all examples in a leaf share the same label).

The CART Algorithm

CART (Breiman et al., 1984)

Input: dataset $S = \{(x_i, y_i)\}_{i=1}^N$.

For each candidate split $t = (f, c)$ (feature f , cutoff c):

$$S_L^t = \{(x, y) \in S : x_f \leq c\}, \quad S_R^t = \{(x, y) \in S : x_f > c\}.$$

For regression, define the **impurity** (squared-loss):

$$L(S_L^t) = \frac{1}{|S_L^t|} \sum_{(x,y) \in S_L^t} (y - \bar{y}_{S_L^t})^2, \quad \bar{y}_S = \frac{1}{|S|} \sum_{(x,y) \in S} y.$$

Pick split $t^* = \arg \min_t L(S_L^t) + L(S_R^t)$. Create child nodes and recurse.

Enumerating splits: for each feature, sort the data points and consider all midpoints between adjacent distinct values.

Stopping Criteria

- All leaf nodes contain only one label y (pure), OR all have identical x (predict the mode/mean).
- Tree hits a maximum depth.
- Each leaf hits a minimum number of points.
- No split improves impurity — **BUT this is a bad criterion!**

△ Trees are myopic — don't stop early based on impurity!

Decision trees greedily select the locally best split at each node **without lookahead**. A split that appears useless now may enable an excellent subsequent split.

Classic example: XOR. Data points at $(0,0), (1,1)$ are class 0; $(0,1), (1,0)$ are class 1. The first split on either feature does not improve impurity (both halves are still 50/50). But after the first split, the second split perfectly separates the classes.

Properties of Decision Trees

CART Properties

Advantages:

- Low memory usage, low computational cost.
- Very fast at test time (just follow the path from root to leaf).
- Handles mixed data types naturally (numerical, categorical, ordinal).

- Invariant to monotone transformations of inputs (only thresholds change).
- Handles missing values well.

Disadvantage: generalisation accuracy is **not competitive** with other methods as a standalone classifier. But they become very powerful through ensembles (bagging, boosting).

Complexity: finding the minimum-size tree is NP-hard; CART's greedy splitting is a heuristic.

Decision Regions

The regions defined by a tree are **Voronoi-like axis-aligned rectangles**. Within each rectangle, the prediction is constant (mean for regression, mode for classification). The tree is essentially a **histogram estimator** of the regression surface.

Part III

Bagging & Random Forests

Motivation: Reducing Variance

Full-depth trees have low bias but high variance. We want $h_{\mathcal{D}} \rightarrow \bar{h}$. By the **weak law of large numbers**, for i.i.d. random variables h_i with mean $\bar{h}$:

$$\hat{h} = \frac{1}{m} \sum_{i=1}^m h_i \rightarrow \bar{h} \quad \text{as } m \rightarrow \infty.$$

Problem: we only have *one* dataset.

The Bootstrap

Bootstrap Sampling

Simulate drawing from $p(X, Y)$ by drawing **uniformly with replacement** from $\mathcal{D}$. Let $q(X, Y|\mathcal{D})$ be the empirical distribution. Sample “new” datasets $\mathcal{D}_i \sim q$ with $|\mathcal{D}_i| = N$. Each bootstrap sample includes $\sim 63\%$ of unique original points (since $P(\text{excluded}) = (1 - 1/N)^N \approx 1/e \approx 0.37$).

Bagging Algorithm

Bootstrap Aggregating (Bagging)

1. Sample m datasets $\mathcal{D}_1, \dots, \mathcal{D}_m$ with replacement from $\mathcal{D}$ (each $|\mathcal{D}_i| = N$).
2. For each $\mathcal{D}_i$, train a model h_i .
3. Predict: $\hat{h}(x) = \frac{1}{m} \sum_{i=1}^m h_i(x)$.

Benefits: reduces variance, provides uncertainty estimates (look at variation among $h_i(x)$), provides an unbiased test-error estimate (OOB error).

Out-of-Bag (OOB) Error

OOB Error — Free Test Error Estimate

For each data point index i , let $S_i = \{k \mid (x_i, y_i) \notin \mathcal{D}_k\}$ be the set of models that did *not* train on point i . Define:

$$\tilde{h}(x_i) = \frac{1}{|S_i|} \sum_{k \in S_i} h_k(x_i).$$

Then the OOB error:

$$\varepsilon_{\text{OOB}} = \frac{1}{N} \sum_{i=1}^N (\tilde{h}(x_i) - y_i)^2$$

estimates the **test error**, not the training error — because each prediction uses only models that never saw that point during training.

Random Forests

Random Forest Algorithm

Bagged full-depth decision trees with **random feature subsampling**:

1. Sample m bootstrap datasets.
2. For each $\mathcal{D}_i$, train a **full-depth** tree h_i , but at each split **randomly subsample** $k \leq D$ **features** and only consider those for the split.
3. Predict: $\hat{h}(x) = \frac{1}{m} \sum_{i=1}^m h_i(x)$.

Why subsample features? Standard bagging can produce *correlated* trees if one feature dominates (every tree splits on it first). Random subsampling **decorrelates** the trees, increasing the effective variance reduction from averaging.

Hyperparameters: only two — m (number of trees) and k (features per split). Rule of thumb: $k = \sqrt{D}$. Results are very insensitive to these choices.

Very little preprocessing needed: works with heterogeneous types, arbitrary scales, missing values.

Part IV

Boosting

Motivation: Reducing Bias

Shallow trees (limited depth) have high bias but low variance. Can we combine many “weak” learners into one “strong” learner?

Gradient Descent in Function Space

Boosting as Functional Gradient Descent

Build an ensemble $H_T(x) = \sum_{t=1}^T \alpha_t h_t(x)$ iteratively. Define:

$$\ell(H) = \frac{1}{N} \sum_{i=1}^N \ell(H(x_i), y_i).$$

At step t , find h_{t+1} by minimising the loss after a “step”:

$$h_{t+1} = \arg \min_h \ell(H_t + \alpha h).$$

Taylor expansion in function space: $\ell(H + \alpha h) \approx \ell(H) + \alpha \langle \nabla \ell(H), h \rangle$, where $\langle g, h \rangle \approx \sum_{i=1}^N g(x_i) h(x_i)$.

So: $h_{t+1} = \arg \min_h \sum_{i=1}^N r_i h(x_i)$, where $r_i = \frac{\partial \ell}{\partial [H(x_i)]}$ are the **pseudo-residuals**.

Generic Boosting Algorithm

1. Input: loss ℓ , step size α , data $\{x_i, y_i\}$, weak learner algorithm $\mathcal{A}$.
2. Initialise $H_0 = 0$.
3. For $t = 0, \dots, T-1$:
 - (a) Compute pseudo-residuals: $r_i = \partial \ell / \partial [H(x_i)]$.
 - (b) Fit weak learner: $h_{t+1} = \mathcal{A}(\{x_i, r_i\})$.
 - (c) If $\sum r_i h_{t+1}(x_i) < 0$: update $H_{t+1} = H_t + \alpha h_{t+1}$. Else: return H_t .
4. Return H_T .

We don't need to minimise exactly — just find h such that $\sum r_i h(x_i) < 0$ to make progress.

Gradient Boosted Regression Trees (GBRT)

GBRT with Squared-Error Loss

With $\ell(H) = \frac{1}{2} \sum (H(x_i) - y_i)^2$:

$$t_i = -r_i = -\frac{\partial \ell}{\partial [H(x_i)]} = y_i - H_t(x_i) \quad (\text{the residuals}).$$

Key derivation: under assumptions ($\sum h(x_i)^2 = \text{const}$, $\mathcal{H}$ closed under negation):

$$\begin{aligned} \arg \min_h \sum r_i h(x_i) &= \arg \min_h -2 \sum t_i h(x_i) & (t_i = -r_i) \\ &= \arg \min_h \sum t_i^2 - 2t_i h(x_i) + h(x_i)^2 & (\text{add constants}) \\ &= \arg \min_h \sum (t_i - h(x_i))^2. \end{aligned}$$

This is just the **squared-error loss** with “targets” t_i ! At each step, fit the next tree to predict the residuals of the current ensemble.

GBRT Algorithm:

1. $H_0 = 0$.
 2. For $t = 0, \dots, T-1$: set $t_i = y_i - H_t(x_i)$; fit $h_{t+1} = \arg \min_h \sum (h(x_i) - t_i)^2$; update $H_{t+1} = H_t + \alpha h_{t+1}$.
- For a general loss, replace t_i with $-\partial \ell / \partial [H(x_i)]$.

AdaBoost

Setting

Binary classification $y \in \{+1, -1\}$, binary weak learners $h : \mathcal{X} \rightarrow \{+1, -1\}$, exponential loss $\ell(H) = \sum_{i=1}^N e^{-y_i H(x_i)}$.

Instance weights: $w_i = \frac{1}{Z} e^{-y_i H(x_i)}$ with $Z = \sum e^{-y_i H(x_i)}$ (the loss itself).

Finding the Best Weak Learner

AdaBoost Weak Learner Selection

Gradient: $r_i = -y_i e^{-y_i H(x_i)}$.

$$\begin{aligned} h_{t+1} &= \arg \min_h \sum r_i h(x_i) = \arg \min_h \sum (-y_i e^{-y_i H(x_i)}) h(x_i) \\ &= \arg \min_h \sum (-w_i y_i h(x_i)) \\ &= \arg \min_h \left[\sum_{h(x_i) \neq y_i} w_i - \sum_{h(x_i) = y_i} w_i \right] = \arg \min_h \sum_{h(x_i) \neq y_i} w_i. \end{aligned}$$

This is the **weighted classification error** $\varepsilon = \sum_{h(x_i) \neq y_i} w_i$. We just need $\varepsilon < 0.5$ (better than random).

Closed-Form Step Size

AdaBoost Step Size Derivation

Minimise $\ell(H + \alpha h) = \sum e^{-y_i [H(x_i) + \alpha h(x_i)]}$ w.r.t. α :

$$\frac{\partial \ell}{\partial \alpha} = - \sum y_i h(x_i) e^{-y_i [H(x_i) + \alpha h(x_i)]} = 0.$$

Split into correct ($y_i h(x_i) = +1$) and incorrect ($= -1$):

$$-(1 - \varepsilon)e^{-\alpha} + \varepsilon e^{\alpha} = 0 \Rightarrow e^{2\alpha} = \frac{1 - \varepsilon}{\varepsilon} \Rightarrow \alpha = \frac{1}{2} \log \frac{1 - \varepsilon}{\varepsilon}.$$

This closed-form step size is a fairly unusual result — most boosting methods treat α as a hyperparameter.

Weight Update

After each step $H_{t+1} = H_t + \alpha h$, unnormalised weights update as:

$$\hat{w}_i \leftarrow \hat{w}_i \cdot e^{-\alpha y_i h(x_i)}.$$

Correctly classified: multiplied by $e^{-\alpha} < 1$ (downweighted). Misclassified: multiplied by $e^{\alpha} > 1$ (upweighted).

Normaliser updates: $Z \leftarrow Z \times 2\sqrt{\varepsilon(1-\varepsilon)}$.

Combined: $w_i \leftarrow w_i \cdot \frac{e^{-\alpha y_i h(x_i)}}{\sqrt{\varepsilon(1-\varepsilon)}}$.

Training Loss Bound

Exponential Convergence

Since Z equals the training loss, after T iterations:

$$\ell(H_T) = Z = N \prod_{t=1}^T 2\sqrt{\varepsilon_t(1-\varepsilon_t)}.$$

Let $c = \max_t \varepsilon_t < \frac{1}{2}$. Then $\varepsilon_t(1-\varepsilon_t) \leq c(1-c) < \frac{1}{4}$, so:

$$\ell(H_T) \leq N [2\sqrt{c(1-c)}]^T = N(1-4\gamma^2)^{T/2},$$

where $c(1-c) = \frac{1}{4} - \gamma^2$. This decreases **exponentially**.

The training error reaches zero when $\ell < 1$, i.e. after $T > 2 \log N / \log(\frac{1}{1-4\gamma^2}) = O(\log N)$ iterations.

Full AdaBoost Algorithm

AdaBoost Algorithm

1. Initialise $H_0 = 0$, $w_i = 1/N$ for all i .
2. For $t = 0, \dots, T-1$:
 - (a) Find $h = \arg \min_h \sum_{h(x_i) \neq y_i} w_i$.
 - (b) Compute weighted error $\varepsilon = \sum_{h(x_i) \neq y_i} w_i$.
 - (c) If $\varepsilon < \frac{1}{2}$: set $\alpha = \frac{1}{2} \log \frac{1-\varepsilon}{\varepsilon}$; update $H_{t+1} = H_t + \alpha h$; update $w_i \leftarrow w_i \cdot \frac{e^{-\alpha y_i h(x_i)}}{\sqrt{\varepsilon(1-\varepsilon)}}$.
Else: return H_t .
3. Return H_T .

Connections:

Bagging vs. Boosting Summary

	Bagging (RF)	Boosting (AdaBoost/GBRT)
Targets	Variance	Bias
Base learner	High-variance (deep trees)	High-bias (stumps/shallow trees)
Training	Parallel, independent	Sequential, dependent
Weighting	Equal ($1/m$)	Adaptive (α_t)
Data sampling	Bootstrap	Reweighting (AdaBoost) / residuals (GBRT)
Overfitting risk	Low	Higher (can overfit with too many rounds)

Connections:

“Off-the-shelf” method comparison (ESL)

Property	Neural Net	SVM	Trees	MARS	k -NN
Mixed data types	Poor	Poor	Good	Good	Poor
Missing values	Poor	Poor	Good	Good	Good
Robust to outliers	Poor	Poor	Good	Poor	Good
Monotone transform invariant	Poor	Poor	Good	Poor	Poor
Scalability (large N)	Poor	Poor	Good	Good	Poor
Irrelevant inputs	Poor	Poor	Good	Good	Poor
Linear combinations	Good	Good	Poor	Poor	Fair
Interpretability	Poor	Poor	Fair	Good	Poor
Predictive power	Good	Good	Poor	Fair	Good

Trees excel at practical robustness; ensembles (RF, boosting) recover their predictive-power weakness.

Part V

Exam-Style Questions & Lecture Exercises

Exam-Style Questions

Exam Q&A:

Q1 — Bias–Variance (3 marks) **Q:** A deep decision tree achieves 0.5% training error but 25% test error. (a) Which term dominates the test error? (b) Suggest two fixes.

A: (a) **Variance** dominates — the large gap between training and test error indicates overfitting. (b) (1) Use bagging / random forests to reduce variance by averaging many deep trees. (2) Collect more training data, which reduces the sensitivity to any particular dataset.

Exam Q&A:

Q2 — AdaBoost Step Size (5 marks) **Q:** Derive the AdaBoost step size $\alpha = \frac{1}{2} \log \frac{1-\varepsilon}{\varepsilon}$.

A: Minimise $\ell(H + \alpha h) = \sum e^{-y_i[H(x_i) + \alpha h(x_i)]}$. Differentiate w.r.t. α : $\sum (-y_i h(x_i)) e^{-y_i H(x_i)} e^{-\alpha y_i h(x_i)} = 0$. Split into correct ($y_i h(x_i) = 1$) and incorrect ($= -1$) sets: $-(1-\varepsilon)e^{-\alpha} + \varepsilon e^{\alpha} = 0$. Rearrange: $e^{2\alpha} = (1-\varepsilon)/\varepsilon$, so $\alpha = \frac{1}{2} \log \frac{1-\varepsilon}{\varepsilon}$.

Exam Q&A:

Q3 — GBRT Residuals (3 marks) **Q:** In gradient boosted regression trees with squared-error loss, what are the targets for each new tree?

A: The targets are the **residuals** $t_i = y_i - H_t(x_i)$ of the current ensemble. This follows from $t_i = -\partial \ell / \partial [H(x_i)] = -(H_t(x_i) - y_i) = y_i - H_t(x_i)$. Each new tree is fit to predict these residuals, correcting the current ensemble's errors.

Exam Q&A:

Q4 — RF vs. Bagging (2 marks) **Q:** Why does random forests subsample features at each split, beyond what standard bagging already provides?

A: Standard bagging of trees can produce correlated trees if one feature is dominant (all trees split on it first). Random feature subsampling forces different features at each split, **decorrelating** the trees and increasing the effective variance reduction from averaging.

Exam Q&A:

Q5 — OOB Error (2 marks) **Q:** Explain how the out-of-bag error provides a test-error estimate without a held-out set.

A: Each data point x_i is excluded from $\sim 37\%$ of bootstrap samples. We average predictions from only those models that did *not* train on x_i . This gives an estimate equivalent to a held-out prediction, since the models never saw x_i during training.

Lecture Exercises & Worked Solutions

Exercise 1: Second Cross-Term Vanishes

Exam Alert: From Bias–Variance, Slide 14

Exercise: Show that $\mathbb{E}_{x,y}[(\bar{h}(x) - \bar{y}(x))(\bar{y}(x) - y)] = 0$.

Solution:

$$\mathbb{E}_{x,y}[(\bar{h}(x) - \bar{y}(x))(\bar{y}(x) - y)] = \mathbb{E}_x[(\bar{h}(x) - \bar{y}(x)) \cdot \mathbb{E}_{y|x}[\bar{y}(x) - y]].$$

Now $\mathbb{E}_{y|x}[\bar{y}(x) - y] = \bar{y}(x) - \mathbb{E}_{y|x}[y] = \bar{y}(x) - \bar{y}(x) = 0$. Therefore the entire expression is zero. $\square$

The key is that $\bar{h}(x)$ and $\bar{y}(x)$ are both deterministic functions of x (they don't depend on the particular y), so they factor out of the expectation over $y|x$.

Exercise 2: Can Every Dataset Have a Pure Tree?

Exam Alert: From Decision Trees, Slides 8–9

Exercise: Is it always possible (for any dataset) to find a tree where each leaf is pure?

Solution: **Not always.** If two data points share *identical feature vectors* $x_i = x_j$ but have *different labels* $y_i \neq y_j$, then no split can separate them — they will always end up in the same leaf, which cannot be pure.

If all data points with the same x also share the same y , then yes: we can always split until each leaf contains only one class (in the worst case, splitting until each leaf contains a single data point).

Exercise 3: Why Not Stop When No Split Improves Impurity?

Exam Alert: From Decision Trees, Slide 17

Quiz: Why don't we stop if no split can improve impurity?

Solution: Because decision trees are **myopic** — they greedily evaluate one split at a time without lookahead. A split that does not improve impurity *at this level* may enable a much better split at the next level.

XOR example: four points at $(0,0), (0,1), (1,0), (1,1)$ with labels 0, 1, 1, 0. The first split on either x_1 or x_2 produces two subsets each containing one class-0 and one class-1 point — no improvement in impurity. But after this first split, the second split perfectly separates each subset.

Stopping when the current split doesn't help would miss this structure entirely. Instead, use a maximum-depth criterion, post-hoc pruning, or validation-based stopping.